

Mi experiencia: Claude Code + Spec Engineering

Guía rápida para el equipo dev — Jikkosoft AI-First

Diego Trujillo · Etapa 1 · Miércoles 24 jun 2026

Antes de esto pensaba en IA como “autocompletado avanzado”. Lo que cambió: el modelo no es el protagonista, la spec sí lo es. El mejor modelo con una spec mala produce basura. El modelo más barato con una spec bien escrita produce output de producción. Eso lo confirma el experimento interno hermes-exploratory.

Workshop 1 — Claude Code + Hermes · Miércoles 24 jun (1-1.5 hrs)

Las herramientas

Herramienta	Rol
Claude Code (Anthropic)	Agente de código en terminal — ejecuta, edita, crea archivos, razona sobre el código
Hermes (NousResearch)	Capa de orquestación: selecciona el modelo y administra el contexto

Regla del programa: toda interacción con LLMs pasa por Hermes. Claude Code es el ejecutor — Hermes elige el modelo correcto para cada tarea.

Instalación en 2 pasos

```
# 1. Claude Code
# Descargar desde https://claude.ai/code
claude --version # verifica instalación

# 2. Hermes
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh | bash
hermes doctor # verifica conexión a proveedores
hermes setup # conecta Anthropic / DeepSeek / Moonshot
```

El ritual /init — por qué importa

```
# En cualquier repo vacío o existente:  
claude /init
```

Genera el CLAUDE.md base: el contrato AI del proyecto. Sin este archivo, el modelo trabaja sin contexto y alucina convenciones, nombres de columnas, rutas. Con él, cualquier modelo (incluso Haiku) respeta las reglas del proyecto.

- Authority hierarchy (qué puede y qué no puede hacer la IA)
- Naming conventions del proyecto
- Safe/unsafe zones (qué archivos nunca tocar)
- Stack y arquitectura

Mi experiencia: la primera vez que lo corrí en el repo de SILIN generó un CLAUDE.md de 4 KB que luego expandí a 21 KB. Desde entonces el modelo no volvió a inventar nombres de tablas.

Navegación básica en Hermes

```
hermes model # selector visual de modelo  
/model anthropic:claude-sonnet-4-6 # cambiar modelo sin salir  
/new # conversación fresca – evita memory bleed  
hermes config list # ver claves de configuración
```

- Interactivo (hermes): el agente pide confirmación antes de ejecutar
- Autónomo (hermes --auto): ejecuta sin confirmar — usar solo en ramas desechables

Videos y recursos

- [Anthropic YouTube — tutoriales y demos de Claude Code](#)
- [Claude Code docs — quickstart oficial](#)
- [Repositorio Hermes \(NousResearch\)](#)

Workshop 2 — Spec Engineering · Miércoles 24 jun, tarde (1-1.5 hrs)

El hallazgo que lo cambia todo

```
Spec A (~150 palabras) + Opus 4.8 → 72 pts  
Spec C (~480 palabras) + Haiku 4.5 → 88 pts
```

Conclusión: mejorar la spec es 3× más barato y efectivo que subir de modelo. El punto de equilibrio costo/calidad: Spec B + Sonnet.

Los 3 niveles de spec

Nivel	Tamaño aprox.	Resultado típico
Spec A — básica	< 150 palabras	Modelo alucina tablas, ignora constraints
Spec B — equilibrio	300-500 palabras	Punto de equilibrio costo/calidad ✓
Spec C — detallada	500+ palabras	Máxima calidad, mejor con tareas complejas

Checklist de una Spec B efectiva

1. Dominio — qué representa en 2-3 oraciones
2. Scope — lista de entidades/tablas con columnas clave
3. Tech stack — versión de DB, IDs, formato de timestamps
4. Convenciones — naming, columnas de estado, tipo de moneda (ej: centavos)
5. Reglas de integridad — soft delete, ON DELETE, UNIQUE, CHECK constraints
6. Reglas de safe-change — columnas nullable, sin renombrados, índices en FKs
7. Fuera de scope — qué NO generar (evita tablas alucinadas)
8. Entregable esperado — formato exacto (solo SQL, sin prosa, sin fences)

El ejercicio práctico que más me sirvió

Spec A: "Crea una tabla de órdenes con sus productos y precios."
→ el modelo inventa nombres, olvida FKs, ignora soft delete

Spec B: "Tabla `orders` con `id` UUID PK, `status` VARCHAR CHECK(...),
`created\_at` TIMESTAMPTZ DEFAULT NOW(), FK a `customers(id)`.
Soft delete: columna `deleted\_at`. Montos en centavos (INTEGER).
Fuera de scope: pagos, envíos."
→ output limpio, constraints correctos

Spec C: Spec B + queries que deben funcionar + casos de borde
→ output de producción directamente usable

Comparar los tres outputs. El ejercicio dura 20 minutos y convence mejor que cualquier presentación.

Regla práctica

score < 80 → la spec necesita más detalle, no un modelo más caro.

Si el output no es lo que esperabas, antes de cambiar de modelo: agrega el punto del checklist que falta. Casi siempre es el 4, 5 o 7.

Videos y recursos

- [Prompt engineering guide \(Anthropic\)](#)
- [Tutorial interactivo — Jupyter notebooks](#)
- [Experimento interno hermes-exploratory \(acceso interno\)](#)

TL;DR para el equipo

1. Instala: Claude Code + Hermes
2. En cada repo nuevo: `claude /init` → expande el CLAUDE.md generado
3. Antes de pedir cualquier cosa al modelo: escribe la spec (mínimo Spec B)
4. Si el output es malo: mejora la spec, no cambies el modelo
5. Usa Hermes como entrada única — él elige el modelo correcto